PROMPT B – Nihongo-based Journey build prompt (main prompt for non-Latin scripts)

> **Which prompt is this?** This is **PROMPT B**, reverse-engineered from **Nihongo Journey v2.5.0**, with German Journey's architecture. Use it as the **main prompt for non-Latin scripts** (Korean, Chinese, Arabic, Hindi, Greek, Russian, Hebrew, Thai …). Use **PROMPT A** (German-based) as its companion for the exact data model, checked IPA for every word form, the accuracy pipeline and the word-box layout. For Latin-script languages, use PROMPT A as the main prompt and this one as the companion. See `README - WHICH PROMPT TO USE.md`.

Version 2 · 23 Oct 2026. This version was reverse-engineered from the finished apps and replaces the old v1 A-to-Z prompt (now in the `old\` folder inside the German release folder's `prompt\` folder). It is built from four sources:

- **Nihongo Journey v2.5.0.** It is the most complete app and the base for all the others.
- **German Journey v2.2.** Its architecture is used here.
- The owner's four briefs:
  - `GERMAN LANGUAGE PROMPT.pdf`
  - `PROJECT.pdf` (Nihongo)
  - `rewriting journey.pdf`
  - the review rounds of 18–23 Oct
- Every correction made during the build.

Part 1 tells **you** (the owner) how to use it. Part 2 is the prompt itself. Copy everything from the Part 2 heading down into a new Claude Code session.

Part 1 — How to use this prompt

1. **Fill in the placeholders.** Each `[PLACEHOLDER]` is your choice:

Placeholder	Example
`[LANGUAGE]`	French
`[APP NAME]`	Français Journey
`[CODE]`	fr (ISO 639-1)
`[FOLDER]`	`CLAUDE FOR LANGUAGE\FRENCH`
`[REPO]`	janisuchin/francais-journey
`[ACCENT]`	a colour not used by another Journey app (Nihongo vermilion #C23B22, German indigo #4B44D6)
`[COUNTRY]`	France
`[SCRIPT]`	Latin / Cyrillic / Devanagari / Arabic / Hangul …
`[PORT]`	a free local port, e.g. 4190 (Nihongo uses 4173, German 4181)

2. **Open a new Claude Code session in the `CLAUDE FOR LANGUAGE` folder**, so Claude can read the existing apps, and paste Part 2 as your first message.
3. **Build in phases and look at each one.** Ask Claude to stop after each phase and show you screenshots and the live link. Small corrections are far cheaper early on.

Phase	What gets built
A	Skeleton and Journey base: profiles, tabs, settings, tap-to-hear, themes
B	Language pack and engines: transliteration/readings, numbers and counters, grammar
C	Days 1–30, with the full day-chapter format
D	Days 31–90, labs, scenarios and culture
E	Photos, dictionary, readings layer and the accuracy audit
F	Release

4. **Plan for usage limits.** A full build plus audit runs about 60–80 AI agents. Run them in waves of about 8–10. More than about 20 at once hit the session limit twice during Nihongo. If agents stop, say "lets go" after the reset and Claude resumes them; their work is saved. Say "use a workflow" or "ultracode" only if you want larger parallel fan-outs.
5. **Expect three feedback rounds.** Section R ("Owner's standing preferences") already contains every correction from the German and Nihongo rounds, so most of them should not come up again.
6. **A human must have the last word on accuracy.** AI reviewers find a great deal: about 280 fixes in Nihongo v2.5. They are still not native speakers. Before a Play Store release, have a native-speaker teacher spot-check, starting with days 1–15. Nothing is ever marked "approved" without a named human.
7. **Builds and money.** Free Expo cloud builds run out each month, so APKs are built on this PC with `tools/build-android.ps1`. A Play Store update needs a signed AAB.

Part 2 — The prompt (copy from here)

A. Goal

Build **[APP NAME]**, a structured, local-first, 90-day **[LANGUAGE]** course for English speakers. It takes learners from zero to solid A1 and core A2, or the language's equivalent (JLPT N5/N4, HSK 1–2, TORFL A1/A2). Learners study 15, 30 or 45 minutes a day. One Expo / React Native codebase serves Android, iOS and the web.

- Build it in `[FOLDER]`, in a private GitHub repo `[REPO]`.
- **Copy the Journey base from Nihongo Journey** (`CLAUDE FOR LANGUAGE\NIHONGO`, the most complete reference).
- Use German Journey (`CLAUDE FOR LANGUAGE\german-journey`) for its architecture document and its Latin-script solutions: gender colours and the respelling guide.
- Read both apps; never edit them.

The app must be **accurate, honest and beautiful**, and look exactly like the other Journey apps, so a learner who knows one knows them all. Only the language, the accent colour and the content differ.

B. Honesty rules (never break)

- No certificates and no guaranteed level. The About page lists the known limitations.
- No fake test results and no fake approvals. Content status is `auto-checked` or `needs-review`. `approved` needs a named human reviewer, and the validator enforces this.
- Audio is device text-to-speech, and the app says so: "computer-generated voice, not a native-speaker recording".
- Report results faithfully. Never say a push, deploy, build or test passed unless it did. If something was skipped, say so.

C. Stack and setup

Concern	Choice
Framework	Expo SDK 58 (or the current SDK), React Native 0.88, React 19.3, TypeScript. Read `AGENTS.md` and the versioned Expo docs before touching any Expo API. Add packages with `npx expo install`.
Navigation	expo-router (file-based, typed routes): JS tabs and a native stack
State	zustand + `persist` (AsyncStorage; localStorage on the web), with **one storage key per learner** (`<app>/state/<learnerId>`) and versioned `migrate()`
Validation	zod schemas. Content is validated in the CLI, in tests, and at runtime (fail-safe: invalid items are hidden)
Audio	expo-speech TTS with an explicit locale, behind a `TtsProvider`. expo-audio for record-and-compare (never uploaded). Optional Web Speech recognition only behind consent.
Language analysis	An offline analyser for the language, used **only in build scripts**: kuromoji for Japanese, or a lemmatiser / morphology dictionary for others. Never ship it in the app bundle.
Tests	Jest (jest-expo, Ionicons mocked) and Playwright web e2e on `[PORT]`
Web hosting	Cloudflare Workers static assets: `wrangler.jsonc` → `[CODE]-journey`, `wrangler.unlocked.jsonc` → `[CODE]-journey-unlocked`, on the `german-journey.workers.dev` account. `scripts/fix-web-assets.mjs` adds the manifest and icons.
Unlocked edition	`APP_VARIANT=unlocked` + `EXPO_PUBLIC_UNLOCK_ALL=1`. It appends " (Unlocked)" to the name and `.unlocked` to the package id, so both apps install side by side.
Licences	Code: MIT. Content: CC BY-NC 4.0. Third-party data keeps its own licence and is listed in `LICENSE-CONTENT`.
Install quirks	`.npmrc` with `legacy-peer-deps=true` (false expo-router peer conflict). Jest needs `@react-native/jest-preset` and `expo-modules-core` as devDependencies.

npm scripts:

- `check` runs typecheck, lint, format:check, content:validate and jest.
- `build:web`, `build:web:unlocked`, `deploy:web` and `deploy:web:unlocked` build and deploy the two web versions.
- `e2e` runs the Playwright tests.
- `content:index`, `content:validate`, `readings:build` and `glossary:auto` maintain the content.

D. Architecture (module boundaries, from German Journey)

```
presentation  src/app/*  src/components/*        screens, navigation, UI kit
base          src/base/*                         profiles, app config (STORAGE_PREFIX, DONATE_URL =
null)
features      src/features/*                     assemble lessons, tests, reviews, sound-of-the-day
from content + engines
engines       src/core/*                         pure, deterministic, unit-tested — no React, no I/O
                transliteration / romanisation, numbers + counters (with English), conjugation +
grammar tables,
                reading index (word split + readings + flags), quiz generation, answer matching, SRS,
progress
language pack src/lang/[CODE]/tap.ts             the only language-specific UI glue (see F)
content       src/content/schema.ts, validate.ts, index.ts, gloss-gaps.ts, dictionary.ts
services      src/services/*                     TTS, recorder, optional recognition (provider
interfaces)
state         src/state/*                        persisted per-learner store, session, hooks, activity
logger
data          content/**/*.json                  all curriculum — never in code
build tools   scripts/*                          validator, content index, readings layer, auto
glossary, photos, dictionary
audit tools   scripts/audit/*                    as-shown export, chunker, text/sentence romaji probes,
scans, dialogue stats
```

The rest of the architecture:

- **SRS:** SM-2 style and deterministic. Learning steps 1 min / 10 min, graduation 1 day, ease 2.5 (minimum 1.3), "mastered" at 21+ days.
- **Unlocking:** each day needs the previous day. Week 1 day 1 also needs the previous week's test at 80%. Invalid days are skipped, so they never trap a learner. "Unlock all chapters" opens everything, per learner.
- **Accessibility:**
  - every control has a role and a label
  - touch targets are at least 44 px
  - text size 90–150%
  - light, dark and high-contrast themes
  - foreign text tagged with `lang` on the web

E. The Journey base (copy from Nihongo; change only names and colours)

Part	What it must do
Learner profiles	"Who is learning [LANGUAGE]?" with name, optional age and an optional 4-digit PIN (a privacy lock only). Each learner has their own progress, settings and cards. Log off and Switch learner are on Me and in Settings. "Continue where you left off" returns to the exact lesson page. Data from older versions becomes the first learner.
Tap-to-hear	`TapText` (any foreign text), `GlossLine` (headings: script line, transliteration line, then compact word chips "word · translit = meaning"), `RichText` (foreign words inside English become tappable, with their transliteration in brackets). A tap reads the word aloud (setting: "Read aloud on tap"), highlights it in **both** the script and transliteration lines, and opens the word box. Digits belong to the foreign run that follows them ("7 時に" is one run, never 時 alone).
Word box	Shows the word, reading, transliteration, meaning, grammar facts, tappable syllables and an example with transliteration and translation, plus normal and slow audio. It opens under the text when there is room (container ≥ 300 px). In narrow places (table cells, chat bubbles, chips) it opens as a **floating card over the page** (a Modal with a dim backdrop, max width 440, scrolls if long). It closes with ✕ or a tap outside. It must never stretch a cell into a long column.

Part	What it must do
No taps inside buttons	`interactive={false}`: in answer options and pressable cards, foreign text shows its transliteration, but the press reaches the option.
Settings	The same sections in the same order: Learner, Learning (Unlock all chapters), [LANGUAGE] text (translations, transliteration on/off and style), Sound (Read aloud on tap, speeds, voice), Speaking, Display (theme, colour theme, text size, high contrast), Reminders, Privacy, About (About, Credits, donate hidden while `DONATE_URL` is null), Advanced.
Colour themes	`paletteFor(name, isDark, hc)` / `THEME_NAMES`: every Journey theme is selectable, with this app's own `[ACCENT]`. Every transliteration line is gold, italic and semi-bold (`c.translit`), straight after the script line.
Icons	Interface emoji are drawn as outline Ionicons in the app colour. Lesson objectives keep 🎯. Objects use real photos or emoji word tiles, never childish clip-art.
Credits	One page: every source and licence, then every photo with author and licence.
Tabs	Today, Path, Practice, Library and Me (section K).

F. Language pack, readings and transliteration (the language-specific core)

`src/lang/[CODE]/tap.ts` exports:

- `SCRIPT_RE` / `hasScript`
- `tapTokens(text, style)`
- `wordToken(text, reading, en?, vocab?, style)`
- `syllables(reading, style)`
- `exampleFor`
- `meaningOf`
- `phraseMeaning`
- `splitRuns` / `japaneseRuns`-style run splitting

Meaning chain, in order:

1. The hand-written glossary (wins for ambiguous words).
2. Curated sentence segment glosses.
3. The course vocabulary.
4. The analysed layer's dictionary form for conjugated words ("eat (polite past)").
5. Numbers and counters (number → English, number+counter → "3 small animals", ordinals "the 2nd").
6. Single-character data (kanji lab).
7. The 5,000-word dictionary, used only when the match is unambiguous.

A compound is **never** glossed by one of its parts (牛丼 is not "cow"). If its meaning is unknown, it becomes a hand-written entry.

Readings layer (essential for non-Latin scripts; strongly recommended for every language), built **offline** by `scripts/build-readings.ts` into `content/readings/readings.json`:

- **Collect texts.** Every foreign text in the content goes in: whole strings without Latin letters, every foreign run inside English text, and `kana` / `reading` fields.
- **Split and read.** The analyser splits each text into learner-level words and gives each word its **reading in context**:
  - a verb or adjective keeps its endings
  - noun + light verb (する) is one word, flagged for a romaji split
  - number + counter is one word, read by the **counter engine** (三匹 さんびき, 六本 ろっぽん, 7時 しちじ)

- **Flags per word:** `p` particle, `v` dictionary-form verb, `s<n>` noun+light-verb split, plus the base form (for meanings).
- **Reading priority:**
  1. a **FIXED_READINGS** table for words the analyser gets wrong
  2. the counter engine
  3. a whole course word
  4. the dictionary reading for all-kanji compounds
  5. the course reading of a conjugated course verb
  6. the analyser Then **context rules** for known ambiguities (Japanese examples: 雨が降ります ふ; 本を開く ひら / ドアが開く あ; 何 before です なん; 〜中 ちゅう/じゅう; 来ない こ; 十分 after 歩いて じゅっぷん; noun + はいりません = は + いります).
- **Never guess.** A lone character with no context gets **no** reading rather than a guess. A reading that contains script characters is rejected.
- **Text in one script only** (kana-only, or any text the analyser mis-splits) uses a **course-knowledge segmenter**: a best-path search over every reading the course has already checked.
  - What it searches: vocabulary, all curated segment readings, glossary words, the dictionary, every engine conjugation, numbers and counters, particles and endings.
  - Its costs:
    - two nouns side by side +1.5
    - a particle at the very start +3
    - a one-sound word +2
    - an unknown chunk (place names) 3.2+, more if it swallows a particle
    - a stand-alone verb ending 2
  - Runs of a second script (katakana loanwords) are single words.
- **Multi-word sentence segments.** Curated segments that are several words are split for romaji, but only when the words' readings add up exactly to the segment's checked reading. Curated morpheme `parts` win.
- **When to rebuild:** run `npm run readings:build` then `npm run glossary:auto` after **any** content change.

Transliteration rules:

- Non-Latin scripts use one standard romanisation (Japanese: modified Hepburn with macrons).
- Latin scripts get an English-letter respelling with stressed syllables in CAPITALS, plus a legend (German).
- **Join rules** (`joinRomaji`): a trailing geminate mark doubles the next consonant (tanoshikatta).
- **Textbook spacing** (`normaliseRomaji`):
  - te-form + auxiliary apart (tabete imasu, matte kudasai)
  - desu ka
  - tai desu
  - n desu ga
  - nakereba narimasen
  - noun + light verb apart, from the course's own light-verb nouns (benkyō shimasu)
  - names capitalised
  - every sentence capitalised
  - a lone geminate dropped (えっ → e)
- **Particles and endings:**
  - Particles are read as said (は → wa, へ → e, を → o, には → ni wa, ては → te wa).
  - A lone kana named as a sound reads as written (は = ha).

    - Grammatical endings that a glossary lists as separate pieces attach to their word in the romaji line (iki + tai → ikitai).

- **Regression tests:** add one for every engine bug found (Nihongo: `src/lang/ja/__tests__/romaji-audit-test.ts`).

Engines (pure and tested): transliteration, numbers (cardinals, ordinals, prices, time, dates, years, phone, words → digits), counters with an **English column**, conjugation and grammar tables, answer matching, quiz, SRS and progress. For Latin scripts, add a rule-based sentence analyser for the validator (German: agreement, V2, case).

G. Curriculum

- **Structure:** 90 days in 13 weekly modules and 7 phases. Levels 0–9 inside the 90 days and 10–12 as an outlined extension.
- **Weekly checkpoint:** each week ends with a test; pass at 80% to open the next week. The app's default pass mark is 0.8, and **every lesson text must say 80%** (a setting can change it).
- **Week:** an English title, a foreign heading (with transliteration chips) and a topic list with no repeats across weeks.
- **Day:** fields `title`, `titleJp`-style foreign heading, `emoji`, `summary`, measurable `objectives` (🎯), `newWords`, `grammar` ids, `pronunciation` topic ids, analysed sentences, a dialogue, a reading, practice, review words, a quiz and `completion` text.

H. The day chapter (the same format every day; mirrors German Journey)

#	Page	Content
1	**Start**	caption "DAY · WEEK · PHASE · LEVEL"; **emoji + bold title**; GlossLine of the foreign title; summary; a card "Today you will be able to…" with 🎯 bullets; a 📖 card with the day's intro text; the **real-life photo card with its story**; new characters (script languages); a counts line "N new words · M example sentences · K quiz questions · Complete the quiz with 80% to finish the day."; Begin
2	**Pronunciation, every day**	The sound of the day. It is the lesson's own topic if it has one; otherwise it is chosen from the sounds the day's own words contain, rotating so topics don't repeat, with pitch/intonation topics using the lab's checked examples. It shows the title, category and IPA badges, the tip, a diagram, cards for **today's words** with the sound's letters coloured (plus transliteration, meaning and audio), the most common error, "Open in the Pronunciation Lab" and **Say it yourself**.
3	Script pages	(non-Latin) new letters and characters with stroke animation
4	**New words (N)**	One page, a card per word: photo or emoji tile, the word coloured by **type** (Latin languages with gender: by gender — der/die/das), reading, transliteration, meaning, key forms (verb group + polite + te-form; adjective negative/past; noun counter or plural), badges, example with audio, 💡 note, and a colour legend
5	**Grammar, every day**	"NEW GRAMMAR", or "REVIEW — first taught on day N" for earlier rules (shorter). It shows the pattern, summary card, formula, explanation, table and a 📘 link. If the day has no grammar step, it uses the rules its sentences use.
6	Sentences	All of the day's sentences on one page, each word in a box with a role-colour underline, plus grammar links
7	Writing / Numbers / Practice	
8	**Dialogue**	Chat bubbles, speakers alternating left/right, Hide English, play-all, Say it yourself, culture notes. **No photo under the dialogue** (the photo is on the start page).
9	Reading, Review, Quiz, Done	

Every button says "Next: ", and every page caption shows "PAGE NAME · n / total".

I. Dialogue standard (measure it; don't guess)

- **Length:** 12–16 lines per day. Days 1–7 may use 10–12.
- **Density:** words per line must meet German Journey's level for the stage. Measure with `scripts/audit/dialogue-stats.py`, which counts English words per line.

Days	Target (English words / line)
1–15	≥ 6.5
16–30	≥ 7.5
31–45	≥ 9
46–60	≥ 9
61–75	≥ 9.5
76–90	≥ 10.5

No more than 2 lines of 3 words or fewer. - **Each line is a complete, natural thought** (greeting + question, answer + reason, thanks + follow-up). Each dialogue tells one coherent story that fits its title and the lesson. Never write fragments like "Oh — teacher, snow!"; write "Oh, look, teacher! It's snowing again!" - **Language:** use the day's new words and grammar. Simple words beyond the syllabus are fine if every segment is glossed. Keep the register: polite by default, casual only in casual dialogues, keigo only where taught. - **Consistency:** keep facts consistent with the day's quizzes and listening questions (prices, change, times, names), and keep story continuity across days. Keep the ids of lines you keep; new ids are unused. - **Review:** a separate reviewer reads every new line before release.

J. Content rules

- All content is JSON in `content/`, validated by zod plus language checks: `npm run content:validate` must show errors 0. Content that fails is hidden, never shown.
- **Every foreign string has an English meaning for every word.** `src/content/gloss-gaps.ts` and `gloss-coverage-test` fail the build on a single gap.
  - Names say "(first name)" / "(surname)" and places say what they are.
  - Deliberately wrong forms are labelled "(wrong — correct: …)".
  - Hand corrections of the auto glossary go in a hand file that overrides `auto.json`.
- **Curated sentences:**
  - segments with t, r (reading for every script character and digit), pos, gloss, fn for particles, and base/form/parts for conjugated words
  - segments join exactly to the text
  - a literal translation, alternatives, common mistakes with "why", grammar ids
  - a `speak` override when TTS misreads
- **Vocabulary:** word, reading, part of speech, verb group or gender/plural, forms, IPA where checked, English, an example with translation, topic, level, an emoji or photo, and notes.
- **Editing:** edit lesson JSON as text. Never re-serialise whole files, because that hides the real change.

K. Screens (mirror Nihongo exactly)

Screen	Layout
Today	A greeting card (greeting in the language + transliteration + audio, "Good evening, [name]!", faint watermark letter); the unlocked banner; the week-test card; Continue; the lesson card (DAY x OF 90 · WEEK · LEVEL, title,

Screen	Layout
	GlossLine, subtitle, objective, Start); Reviews; Today's goal; Words/Characters card; Labs grid; Your journey
Path	Week cards: phase/week caption, level badges (each opens its level page), English title, foreign heading with chips, distinct topics, day circles, Test pill, progress bar; Explore card; extension levels
Practice	Daily review, Flashcards, Scenario simulator; Labs (pronunciation, script/letters, numbers, sentence explorer, time, speaking); Quiz centre and week tests
Library	Search with special-letter keys; a 2-column grid (Dictionary first, Letters/Script, Placards, Vocabulary, Grammar, Pronunciation, Scenarios, Sentence explorer, Numbers, Time, Explore, Flashcards); Favourites, Recently learned, My lists. **Every word is a card**, never a loose row.
Me	Profile card (avatar, name, "x of 90 days", progress, Switch learner, Log off), 6 progress stats, accuracy by skill, menu rows (report, settings, privacy, about, credits), achievements
Also	Day page, level pages, grammar pages, Dictionary (5,000 words), Sentence explorer, Placards (about 18–20 sets), Scenario pages with photo scenes, Culture chapters, Credits, Settings, Profiles, Onboarding, Report, About

Numbers & counters: a number calculator (digits or words in; the language, transliteration, audio, kanji/words, ordinal and year out). The counters table has columns for number, written form, reading, **transliteration + English** ("1 small animal", "3 o'clock", "the 2nd (date)") and audio. Lesson number pages show the English too.

L. Labs and extras

- **Pronunciation lab:** a page for every sound, with mouth/tongue diagrams, minimal pairs, record-and-compare and a mastery quiz.
- **Script lab** (non-Latin): every character with stroke order animation, tracing and example words.
- **27 real-life scenarios:**
  - airport, transport, tourist info, hotel, restaurant, supermarket, clothes and electronics
  - home, school, office, bank, pharmacy, hospital, directions, help, friends, phone, appointments
  - renting, public services, weather and plans, festival and museum, job interview, machines, etiquette, dietary
  - each with a photo scene, dialogue, branching role-play, practice, common mistakes and a quiz
- **Explore [COUNTRY]:** about 8–10 culture chapters with real photos.
- **"In real life" photo card** for every day.
- **Placement check, week tests, a cumulative test and a final exam.**

M. Photos

- **Source:** licensed Wikimedia Commons only (public domain, CC0, CC BY or CC BY-SA), fetched with `scripts/commons_photo.py` at about 1 request per second, square-cropped to 360×360 and credited. Each photo carries a © badge.
- **Scene photos:** up to 15 objects each, each with a red outline box (check with `render-spots.py`). An object is shown on hover/touch or when its word chip is tapped, on the same page, with no numbers.
- **Word photos:** about 150–300 square photos. Other words get an emoji tile or a letter tile.

N. Accuracy process (before every release; the Nihongo v2.5 method)

1. **Automatic checks:** validator errors 0, gloss gaps 0, regression tests and a full-sentence romaji scan (`scripts/audit/scan.ts`).
2. **Export as shown.** `scripts/audit/audit-dump.ts` exports every part (lessons by 15 days, grammar, vocab, kanji/script, glossary, auto glossary, scenarios/culture/placards/real-life) exactly as learners see it, **romaji included**. `chunk.py` splits the export into chunks of about 450 KB.

3. **Two independent reviewers per chunk.** They share a brief (`REVIEW_BRIEF.md`) and write a JSON findings file: file, where, current, problem, category (grammar, reading, romaji, translation, gloss, fact, engine), severity, fix. Systematic romaji problems are reported once, as "engine".
4. **A third reconciler per area:**
   - Fixes both reviewers agree on are applied.
   - Single-reviewer findings are judged, and applied only if real.
   - Engine findings are checked against the **current** output (`rt.ts` for any text, `sr.ts` for a sentence id); the engine is fixed in code, with a regression test, never patched in content.
   - Every decision is recorded.
5. **Rebuild and re-check:** `readings:build` → `glossary:auto` → validator → gloss gaps 0 → `npm run check` → e2e.
6. **Waves:** run reviewers in waves of 8–10 agents. Re-export after big engine changes so reviewers don't report issues that are already fixed.

O. Tests and layout

- **Unit tests:** engines, content, components, gloss coverage and romaji regressions.
- **E2E (Playwright, web build):**
  - onboarding, profiles and switching
  - the lesson flow (page 2 is Pronunciation), quiz and persistence
  - flashcards, scenarios and placards
  - the word box (floating card from a table cell, closes on an outside tap)
  - the counters' English
  - dictionary, explorer, calculator, settings and themes
- **Layout sweep:** every screen at 320×568, 344×882, 360×640, 360×800 and 800×1280, and at the largest text size, with no off-screen text and no horizontal page scroll.
- **Visual check:** screenshot every changed screen and look at it before reporting. Use "Unlock all chapters" to reach later days.

P. Release (every version; never overwrite)

1. Bump the version in `app.json` and `package.json`.
2. `npm run check` green and e2e green, then commit, tag `v<x.y.z>` and push.
3. Deploy web and unlocked web. Open both live links and check the live bundle contains the new content: normal locks Day 2, unlocked opens it.
4. Build the Android APK and the unlocked APK (EAS if the quota allows, otherwise `tools/build-android.ps1 -Variant normal|unlocked`). Check each APK's package id, version, and that its bundle contains the new content.
5. File everything in `all release journey and links and architecture\releases\v<x.y.z> (<date>, Expo SDK <n>)\`:
   - android/
   - android-unlocked/
   - web/ and web-unlocked/ (a `.url` shortcut and an offline zip each)
   - expo-sdk58-source/ (a `git archive` of the tag)
   - README.txt
6. Update the folder's README (Latest version table), RELEASE_JOURNEY.md (newest first) and the plan/requirements document (one lettered section per owner review). Commit and push. APKs and zips are git-ignored.
7. Older versions stay in their own folders; only the two web links always show the newest version.

Q. Security and git

- Never commit `.env` files, keys, certificates, store credentials, recordings or learner data.
- Never ask for passwords, and never paste tokens into files.
- The repo is private. Commit messages end with the co-author line.

R. Owner's standing preferences (from the German and Nihongo builds)

Reporting

- The **clickable live web link** goes at the top of every final report.
- Always push to GitHub after changes, and always produce the four builds plus the source in a new version folder.
- Report in plain English: what changed, how it was checked, the links and the files. Explain *why* when something differs from the reference. Ask only when a decision is genuinely the owner's.

Format

- The same format across all Journey apps. Compare with German Journey page by page when asked. "Formatting only" means change nothing else.

Readings and tap boxes

- Every foreign heading has transliteration, word chips with meanings, and audio.
- Every foreign word anywhere can be tapped, heard and understood. Read aloud and highlight must match.
- The word box never sends the learner to another page, and never stretches a table cell.
- Words are cards, not loose rows. Objects use real photos (or emoji tiles), never childish pictures.

Lessons

- Day chapters follow section H exactly: page 2 is always Pronunciation, a grammar page every day, the photo on the start page.
- Dialogues are long **and dense** (section I), natural and sensible.
- Lesson objectives keep 🎯.

Accuracy

- Grammar, transliteration and translation are checked by multiple AI reviewers who cross-check each other, before a release.

S. Pitfalls already met (avoid them)

- **Romaji from an analyser that guesses.** Dictionary-only greedy splitting gave "toki ni okimasu" for 7時に, "sanhiki", "hairimasen" and "ha" for は. Use the readings layer (F) from the start.
- **Analysers fail on one-script text.** kuromoji split きょうはそら… into き + …; use the course-knowledge segmenter.
- **A whole-text fallback that is too eager** hides good splits. Fall back only when no split exists.
- **Python heredocs eat backslashes** in regexes. Write patch scripts to files, and use `assert` on every replacement so a silent non-match is caught.
- **In JS template strings `\s` becomes `s`** and `\b` becomes a backspace. Write `\\s` and `\\b`.
- **Agents share the scratchpad.** Give each agent its own subfolder.
- **Session limits.** Too many parallel agents stop together. Resume each with a short message; their work is kept.
- **Dumps can mislead.** An export that joins words differently from the app makes reviewers report non-issues. Make the export use the app's own functions.
- **Single-segment phrases** (そうですか as one segment) need the splitter or the normaliser, or their romaji fuses.

Z. Acceptance checklist (all must be true before "done")

- [ ] All 90 days, 13 week tests, 27 scenarios, the labs, dictionary, explorer and calculator work in web, unlocked web, Android and unlocked Android.
- [ ] Every day follows section H. Page 2 is Pronunciation. There is a grammar page every day. Every dialogue passes `dialogue-stats.py` for its stage.
- [ ] Every foreign word on every screen can be tapped, heard and understood. Gloss gaps are 0. The romaji scan is clean.
- [ ] Validator 0 errors, unit tests green, e2e and layout sweep green.
- [ ] The two-reviewer + reconciler audit is done, its fixes are applied and recorded, and there are no fake approvals.
- [ ] Release folder complete, version tagged and pushed, both live links verified, both APKs verified.

References:

- Nihongo Journey v2.5.0: https://nihongo-journey.german-journey.workers.dev (repo janisuchin/nihongo-journey; architecture in `NIHONGO\docs\JOURNEY_BASE.md` and `NIHONGO\all release journey and links and architecture\`).
- German Journey: https://german-journey.german-journey.workers.dev (repo janisuchin/german-journey; architecture in `german-journey\docs\ARCHITECTURE.md`).